



中国科学技术大学

高级编程与性能优化

吴锋

Email: wufeng02@ustc.edu.cn



本章内容概述

- ❖ 位运算
- ❖ 宏操作（带参数、系统宏）
- ❖ 静态与动态库
- ❖ 编程优化方法



本章内容概述

- ❖ 位运算
- ❖ 宏操作（带参数、系统宏）
- ❖ 静态与动态库
- ❖ 编程优化方法



位运算的概念

- ❖ 很多算法是按二进制位进行运算的
- ❖ 位运算速度快，效率高
- ❖ 位运算的运算对象是**二进制**的位
- ❖ 只能对**整型数据**（包括字符型）进行位运算
- ❖ 负数以补码形式参与运算



位运算符

❖ 注意位运算符与逻辑运算区别

运算符	运算	举例	优先级从高到低
			(逻辑非运算符)
~	按位取反	~flag	(算术运算符)
<<	左移	a << 2	
>>	右移	b >> 3	
			(关系运算符)
&	按位与	flag & 0x37	(赋值运算符)
^	按位异或	flag ^ 0xC4	
	按位或	flag 0x5A	



按位与 (Bitwise AND) &

❖ 运算规则

↻ $0 \& 0 = 0;$

↻ $0 \& 1 = 0;$

↻ $1 \& 0 = 0;$

↻ $1 \& 1 = 1;$

❖ 特殊用法

↻ 特定位清零

↻ 保留其它位

	1010, 1101	(0xAD)
&	0110, 1001	(0x69)
	0010, 1001	(0x29)

	xxxx, xxxx	
&	0110, 0010	(0x62)
	0xx0, 00x0	



按位或 (Bitwise Inclusive OR) |

❖ 运算规则

↻ $0 | 0 = 0;$

↻ $0 | 1 = 1;$

↻ $1 | 0 = 1;$

↻ $1 | 1 = 1;$

	1010, 1101	(0xAD)
	0110, 1001	(0x69)
	1110, 1101	(0xED)

❖ 特殊用法

↻ 特定位置一

↻ 保留其它位

	xxxx, xxxx	
	0110, 0010	(0x62)
	x11x, xx1x	



按位异或 (Bitwise Exclusive OR) ^

❖ 运算规则

$$\text{0} \wedge \text{0} = \text{0};$$

$$\text{0} \wedge \text{1} = \text{1};$$

$$\text{1} \wedge \text{0} = \text{1};$$

$$\text{1} \wedge \text{1} = \text{0};$$

	1010, 1101	(0xAD)
^	0110, 1001	(0x69)
	1100, 0100	(0xC4)

❖ 特殊用法

⌘ 特定位取反

⌘ 保留其它位

	xxxx, xxxx	
^	0110, 0010	(0x62)
	x $\bar{x}$ x $\bar{x}$, x $\bar{x}$ x $\bar{x}$	



按位取反 (Bitwise NOT) ~

❖ 运算规则

$$\sim 0 = 1;$$

$$\sim 1 = 0;$$

~	0110, 1001	(0x69)
	1001, 0110	(0x96)



左移 (Left Shift) $\ll$

❖ 运算规则

↻ $i \ll n$

↻ 把 i 各位全部向左移动 n 位, 不会改变 i 的值

↻ 最左端的 n 位被移出丢弃

↻ 最右端的 n 位用 0 补齐

$5 \ll 3 = 40$

$00000101 \text{ (0x05)} \ll 3 \rightarrow 00101000 \text{ (0x28)}$

❖ 用法

↻ 若没有溢出, 则左移 n 位相当于乘上 2^n

↻ 运算速度比真正的乘法和幂运算快得多



右移 (Right Shift) >>

❖ 运算规则

∞ $i >> n$

∞ 把 i 各位全部向右移动 n 位, 不会改变 i 的值

∞ 最右端的 n 位被移出丢弃

∞ 最左端的 n 位用 0 补齐(逻辑右移)

∞ 或最左端的 n 位用符号位补齐(算术右移)

} 由编译系统的实现者决定。
为了可移植性, 最好仅对无
符号数进行移位运算

$5 >> 2 = 1$

❖ 用法

$00000101 \ (0 \times 05) >> 2 \rightarrow 00000001 \ (0 \times 01)$

∞ 右移 n 位相当于除以 2^n , 并舍去小数部分

∞ 运算速度比真正的除法和幂运算快得多



位运算的规则

- ❖ 按位运算时，两个操作数长度应相等，
- ❖ 否则先扩展，再运算
 - ☞ 两个操作数右端对齐
 - ☞ 短的数据左端用符号位补齐
 - ❖ 正数或无符号数左端用 0 补满
 - ❖ 负数左端用 1 补满
- ❖ 位运算符都不会改变参与运算变量的值
- ❖ 复合赋值运算符（改变参与运算变量的值）
 $\&=$, $\wedge=$, $|=$, $\sim=$, $\ll=$, $\gg=$



位运算 (例一)

- ❖ 例：如何判断一个 `int` 型变量 `n` 的第7位（从右往左，从 0 开始数）是否是 1？
- ❖ 答：只需看 `n & 0x80` 的值是否等于 `0x80` 即可。

XXXX XXXX XXXX XXXX XXXX XXXX **XXXX** XXXX (n)

& 0000 0000 0000 0000 0000 0000 **1**000 0000 (0x80)

0000 0000 0000 0000 0000 0000 **x**000 0000



位运算（例二）

❖ 例：将 16 进制短整数按二进制打印输出

☞ 输入：F1E2

☞ 输出：1111000111100010

☞ 输入：13A5

☞ 输出：0001001110100101

```
#include <stdio.h>

int main() {
    int i;
    short a;
    scanf("%X", &a);
    for (i=15;i>=0;i--)
        printf("%1d",
                a&1<<i ? 1 : 0);
    return 0;
}
```



异或运算的特点

❖ 异或运算的特点是：

∞ 如果 $a \oplus b = c$ ，那么就有 $c \oplus b = a$ 以及 $c \oplus a = b$

❖ 此规律可以用来进行最简单的加密和解密以及信息校验

∞ 加密：明文 $\oplus$ 密钥 = 密文；解密：密文 $\oplus$ 密钥 = 明文

∞ 校验：信息 (a b c d) + 校验码 ($a \oplus b \oplus c \oplus d$)

❖ 还可以用来交换两位数 a 和 b

∞ $a = a \oplus b$

∞ $b = a \oplus b$ ，即： $(a \oplus b) \oplus b = a$

∞ $a = a \oplus b$ ，即： $(a \oplus b) \oplus a = b$